CareCall 제작 매뉴얼

ESP32-C3 호출 단말 · Raspberry Pi 5 서버

설치 · 설정 · 시험 · 운영 절차

목차

아래 순서대로 설치하고 각 단계의 정상 결과를 확인한다.

1. 빈 Raspberry Pi 용 통합 OS 설치
2. 설치 전에 준비할 값
3. 하드웨어 연결
4. Raspberry Pi 에 소스 준비
5. Raspberry Pi 기본 환경 설치
6. Raspberry Pi 프로그램과 초기 설정 배치
7. UFW 방화벽 설정
8. 초기 설정 점검
9. MQTT listener 와 CareCall 서비스 활성화
10. Pi Wi-Fi 관리자 최초 활성화
11. Telegram 운영자와 보호자 등록
12. ESP32-C3 펌웨어 준비
13. ESP32-C3 빌드 및 플래시
14. ESP32-C3 최초 Wi-Fi 설정
15. 전체 기능 시험
16. 설치 장소에서 Wi-Fi 변경
17. 호출 버튼 추가
18. 장애 확인 명령
19. GitHub 에 올리지 않는 파일
20. 참고 자료

0. 문서 목적

이 문서는 새로운 Raspberry Pi 5 와 새로운 ESP32-C3 Super Mini 에 CareCall 시스템을 설치하고, 호출 버튼 → Raspberry Pi → Telegram 보호자 알림 → 보호자 확인 → LED 소등까지 동작시키는 절차를 설명한다.

기준 소스는 다음 GitHub 커밋이다.

- 저장소: https://github.com/elecage/DU_AT/tree/3767ab7313b188de9ffed64f212d0d080aa1a97b/Carecall_Button/2026_10

설치 대상 구성은 다음과 같다.

구분	기준
Raspberry Pi	Raspberry Pi 5
운영체제	Ubuntu Server 24.04.5 LTS, arm64
ESP32	ESP32-C3 Super Mini, Flash 4 MB
ESP-IDF	v5.4.1
MQTT	Mosquitto, TCP 1883, 계정 인증 및 ACL 적용
서버 프로그램	Python, paho-mqtt 2.1.0
데이터베이스	SQLite
보호자 인터페이스	Telegram Bot
기본 호출 버튼	button01, button02, button03
기본 장치 세트	set01

1. 빈 Raspberry Pi 용 통합 OS 설치

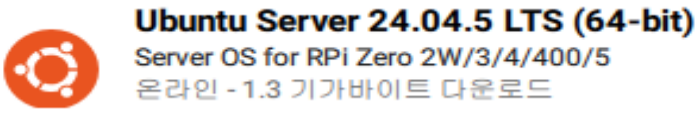
A. **"SD 카드"를 준비한다**

B. https://www.raspberrypi.com/software/ 해당 사이트에서 Imager 를 설치한다.

C. **장치에서 "Raspberry Pi 5"를 선택한다.**



D. **"Other General-purpose OS"를 선택 후 "Ubuntu"를 선택한 후 아래 사진 글 선택.**



E. **"SD 카드"에 설치 후 시작하면 된다.**

2. 설치 전에 준비할 값

설치 중 다음 값을 입력해야 한다. 비밀번호와 토큰은 GitHub, 보고서, 화면 캡처에 기록하지 않는다.

항목	예시 또는 조건
초기 공유기 SSID	Pi 와 ESP32 가 함께 사용할 2.4 GHz Wi-Fi
초기 공유기 비밀번호	공유기 실제 비밀번호
Pi 설정 AP 이름	CareCall-Pi-로 시작하는 32 자 이내 이름
Pi 설정 AP 비밀번호	영문·숫자만 사용한 16~63 자
ESP32 설정 AP 비밀번호	영문·숫자만 사용한 16~63 자
MQTT 수신기 비밀번호	영문·숫자·_·-만 사용한 16~63 자
button01 MQTT 비밀번호	위와 같은 형식, 다른 계정과 중복 금지
button02 MQTT 비밀번호	위와 같은 형식, 다른 계정과 중복 금지
button03 MQTT 비밀번호	위와 같은 형식, 다른 계정과 중복 금지
Telegram Bot 사용자명	@를 제외한 사용자명, 마지막이 bot
Telegram Bot Token	BotFather 에서 발급한 토큰
ESP32 연결 포트	Windows 장치 관리자에서 확인한 COM3, COM4 등의 포트

MQTT 비밀번호 네 개는 Pi 설정과 각 ESP32 빌드 설정에 같은 값을 입력해야 하므로 안전한 비밀번호 관리자에 별도로 보관한다.

3. 하드웨어 연결

3.1 ESP32-C3 호출 버튼

- 호출 버튼은 NO 방식 버튼을 사용한다.
- 버튼 한쪽은 ESP32-C3 의 GPIO4 에 연결한다.
- 버튼 다른 쪽은 GND 에 연결한다.
- 펌웨어는 내부 Pull-up 을 사용하며 버튼이 눌리면 LOW 로 인식한다.

3.2 NeoPixel LED 링

- WS2812B 16 픽셀 LED 링의 DI 를 ESP32-C3 의 GPIO2 에 연결한다.
- LED 링 전원과 GND 를 연결한다.
- ESP32-C3 와 LED 링은 GND 를 공통으로 연결한다.
- 기본 밝기는 16/255 이다.

3.3 전원부

- 충전식 배터리와 VLT-CP013 충전 모듈을 사용한다.
- LX-LBCE1 슬라이드 스위치는 장치의 물리 전원 ON/OFF 용도로 사용한다.
- USB 는 펌웨어 기록 또는 충전 용도로 사용한다.
- 이 장치는 Wi-Fi 연결을 계속 유지하므로 Deep-sleep 을 사용하지 않는다.

3.4 부품리스트

번호	부품명	모델명 및 규격	수량	용도
1	마이크로컨트롤러	ESP32-C3 SuperMini	1 개	시스템 제어 및 무선통신
2	아케이드 푸시 버튼	30mm Push Button	1 개	보호자 호출 입력
3	RGB LED 링	WS2812 계열, 5V (추정)	1 개	호출 및 시스템 상태 표시
4	무선충전 수신 모듈	QR100_YJX (기판 표기)	1 개	무선충전 전력 수신
5	무선충전 수신 코일	수신 모듈 부속 코일	1 개	무선 전력 수신
6	충전·방전 모듈	VLT-CP013	1 개	배터리 충전 및 전원 공급
7	리튬폴리머 배터리	3.7V 5000mAh	1 개	시스템 전원 공급
8	슬라이드 스위치	SPDT 3 핀	1 개	시스템 전원 ON/OFF
9	저항	330~470Ω	1 개	LED 데이터 신호 안정화
10	전해콘덴서	1000μF, 10V 이상	1 개	LED 전원 안정화 (권장)
11	연결 전선	실리콘 전선	적당량	부품 간 전기적 연결

3.5 전체 배선표

번호	출발 부품	출발 단자	연결 대상	대상 단자	기능
1	무선충전 수신 모듈	VOUT	LX-LBCE1	5V 입력	무선충전 전원 공급
2	무선충전 수신 모듈	GND	LX-LBCE1	입력 GND	접지
3	리튬폴리머 배터리	+ (빨간선)	LX-LBCE1	B+	배터리 양극
4	리튬폴리머 배터리	- (검은선)	LX-LBCE1	B-	배터리 음극
5	LX-LBCE1	5V OUT	슬라이드 스위치	COM	전원 스위치 입력
6	슬라이드 스위치	외곽 핀 1	ESP32-C3	5V/VBUS	시스템 전원 공급
7	슬라이드 스위치	외곽 핀 1	RGB LED 링	5V	LED 전원 공급
8	LX-LBCE1	출력 GND	ESP32-C3	GND	공통 접지
9	LX-LBCE1	출력 GND	RGB LED 링	GND	LED 접지
10	ESP32-C3	GPIO4	아케이드 버튼	NO	버튼 입력
11	ESP32-C3	GND	아케이드 버튼	COM	버튼 접지
12	ESP32-C3	GPIO2	330~470Ω 저항	한쪽 단자	LED 제어 신호
13	330~470Ω 저항	반대쪽 단자	RGB LED 링	DI	LED 데이터 입력

3.6 ESP32-C3 핀맵

핀 번호	연결 부품	사용 목적
GPIO2	RGB LED 링 (DI)	LED 색상 및 밝기 제어
GPIO4	아케이드 버튼 (NO)	호출 신호 입력
5V/VBUS	슬라이드 스위치	전원 공급
GND	LX-LBCE1 및 아케이드 버튼	공통 접지

3.6 슬라이드 스위치 핀맵

단자	연결 대상	기능
COM (가운데)	LX-LBCE1 5V OUT	전원 입력
외곽 핀 1	ESP32-C3 및 RGB LED 링 5V	전원 출력
외곽 핀 2	연결하지 않음	미사용

3.7 조립 가이드

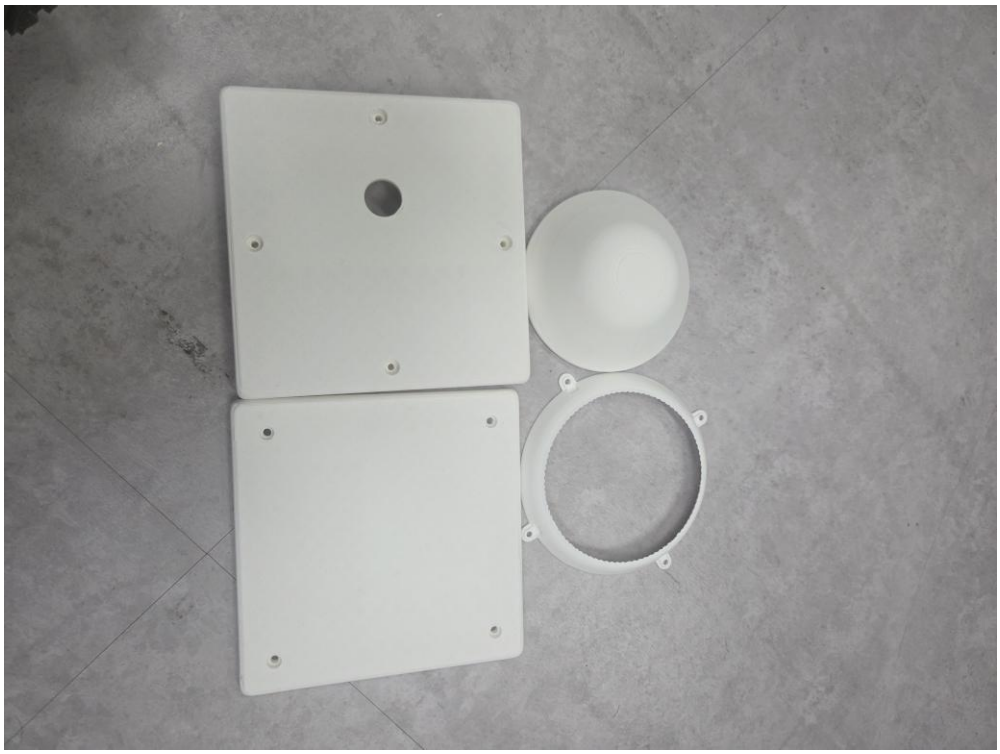


그림 1 케이스 전체 부품

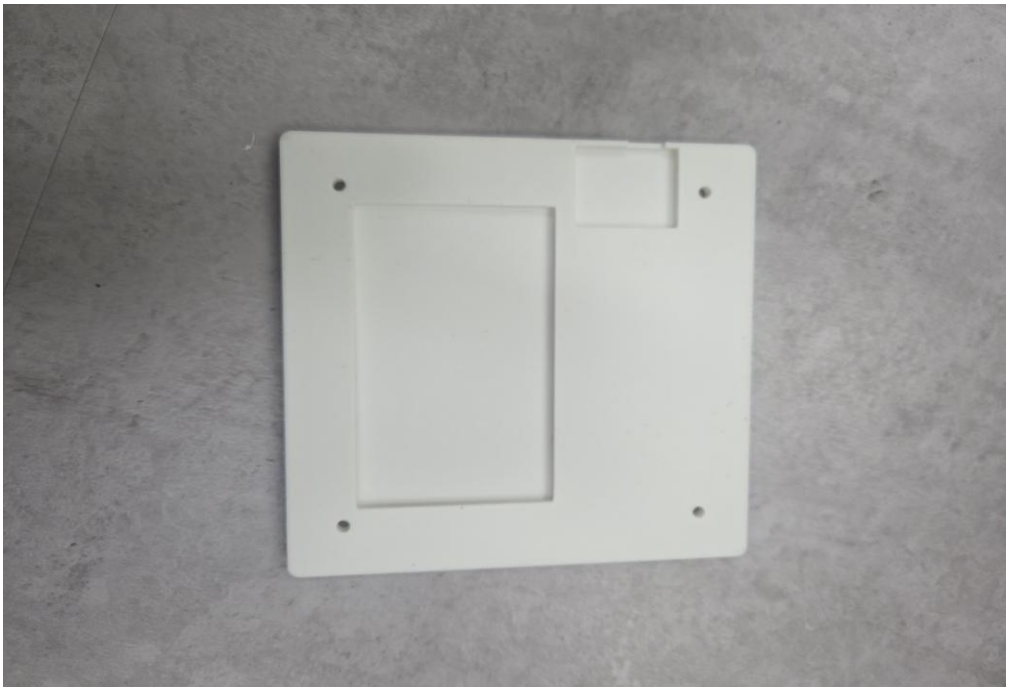


그림 2 밑판을 준비해주시고

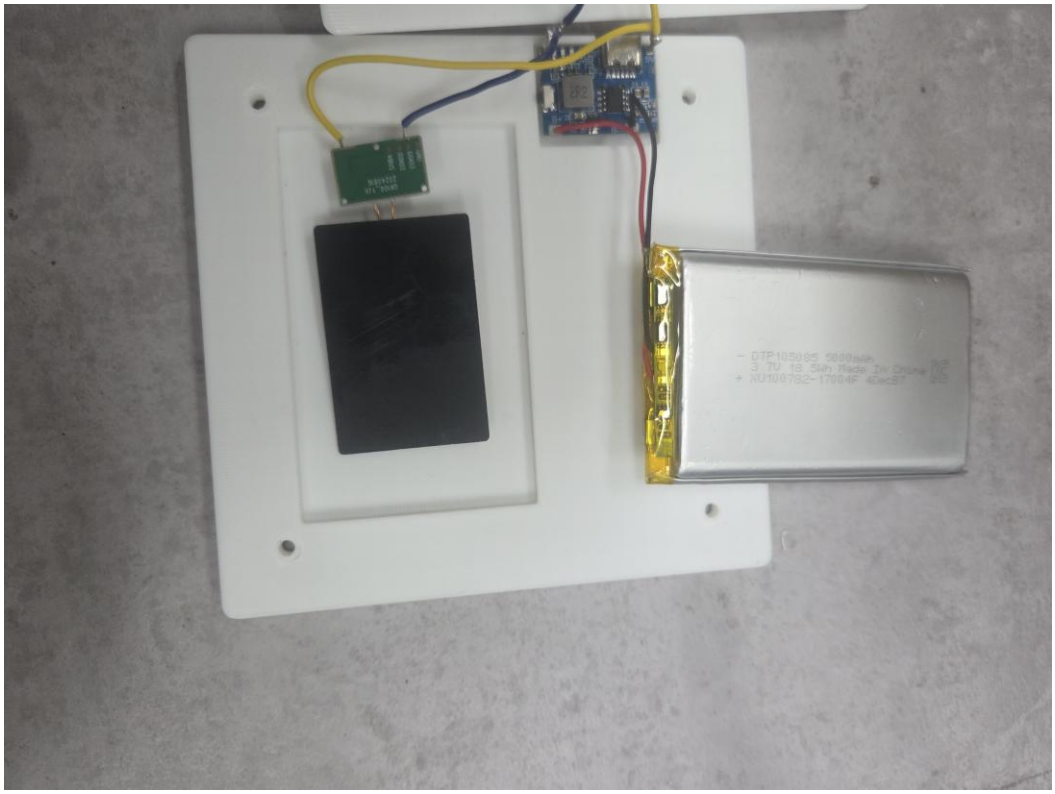


그림 3 부품을 배치해주세요



그림 4 다 배치하면 이런 모양이 나옵니다

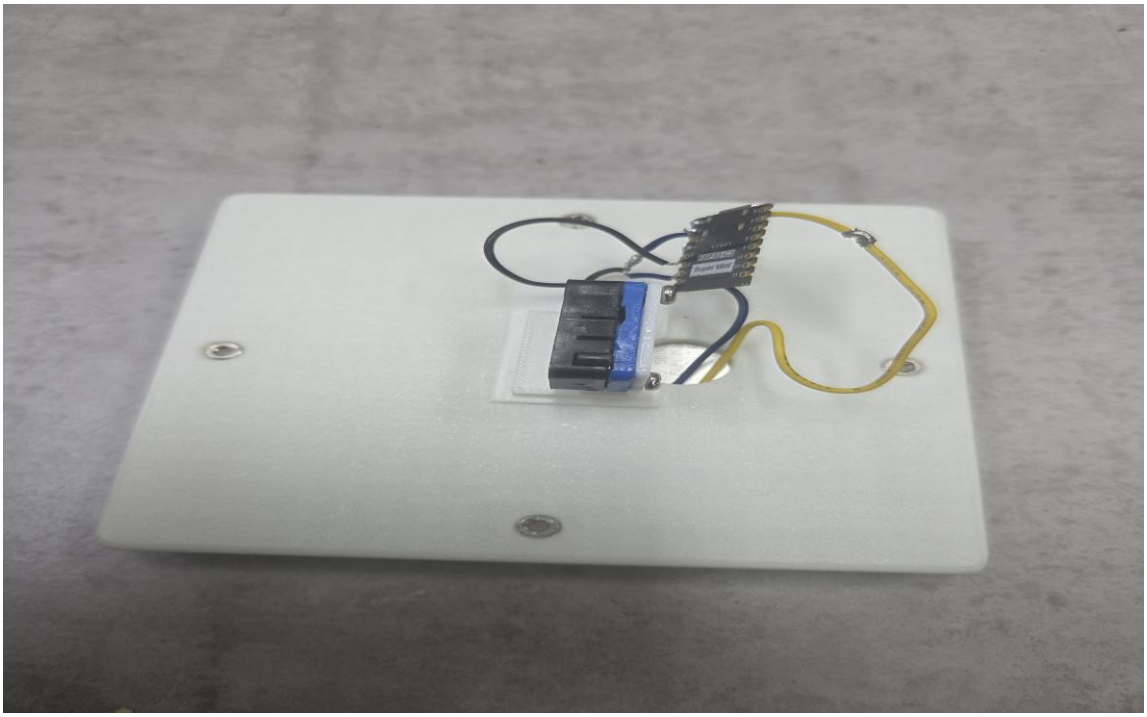


그림 5 상판을 결합한 뒤 나사를 조이고 나머지 부품도 배치해주세요

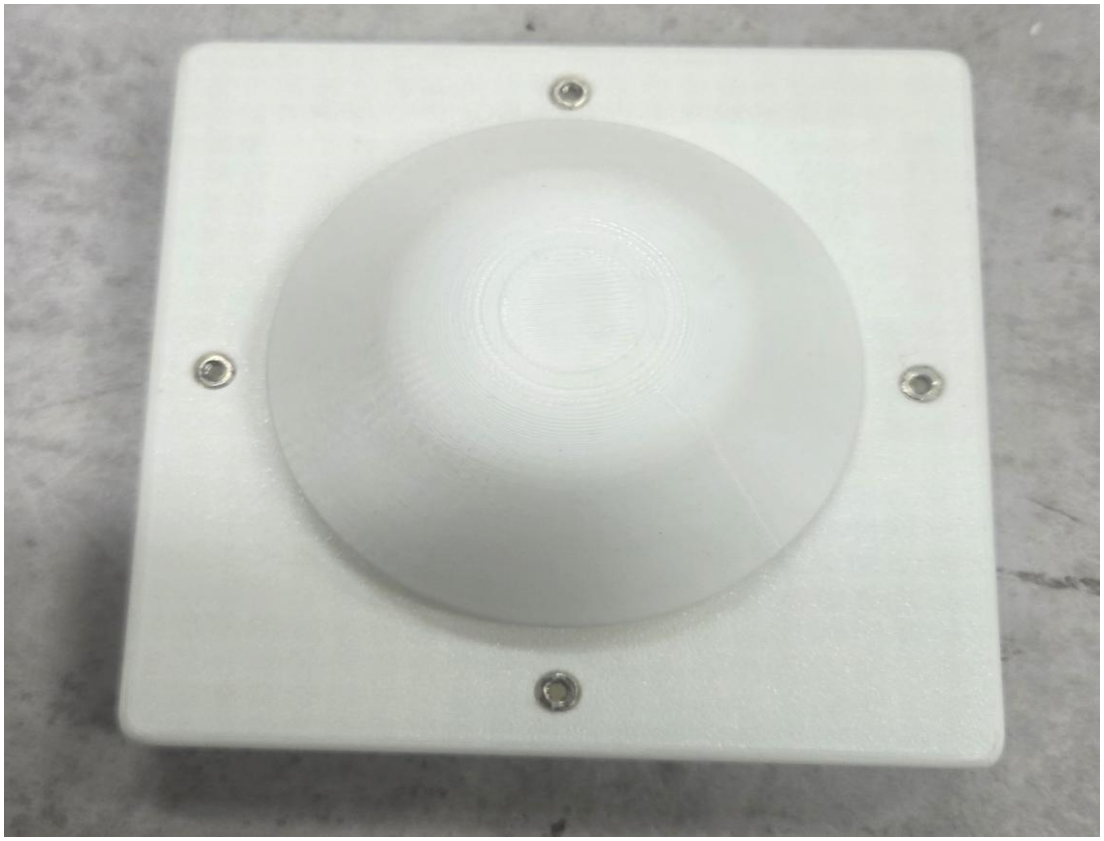


그림 6 뚜껑을 얹어 주세요

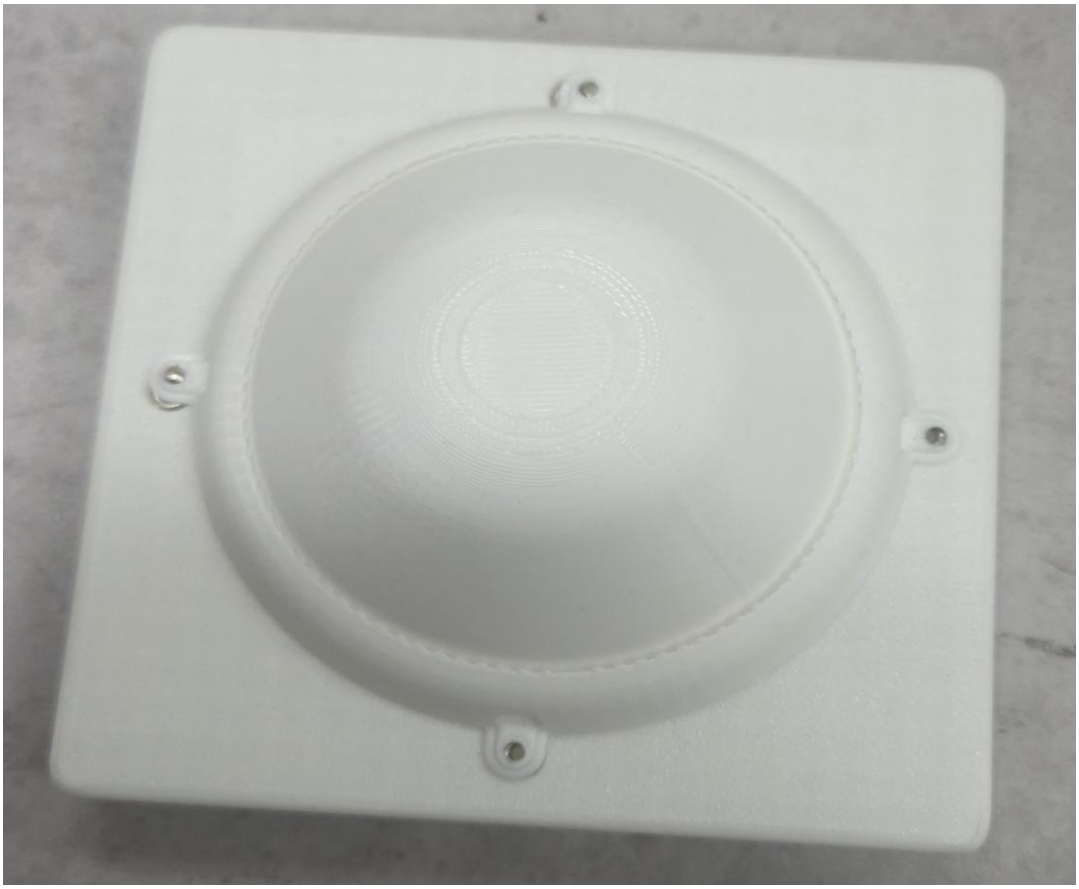


그림 7 뚜껑 틀을 배치하고 나사로 조여주세요

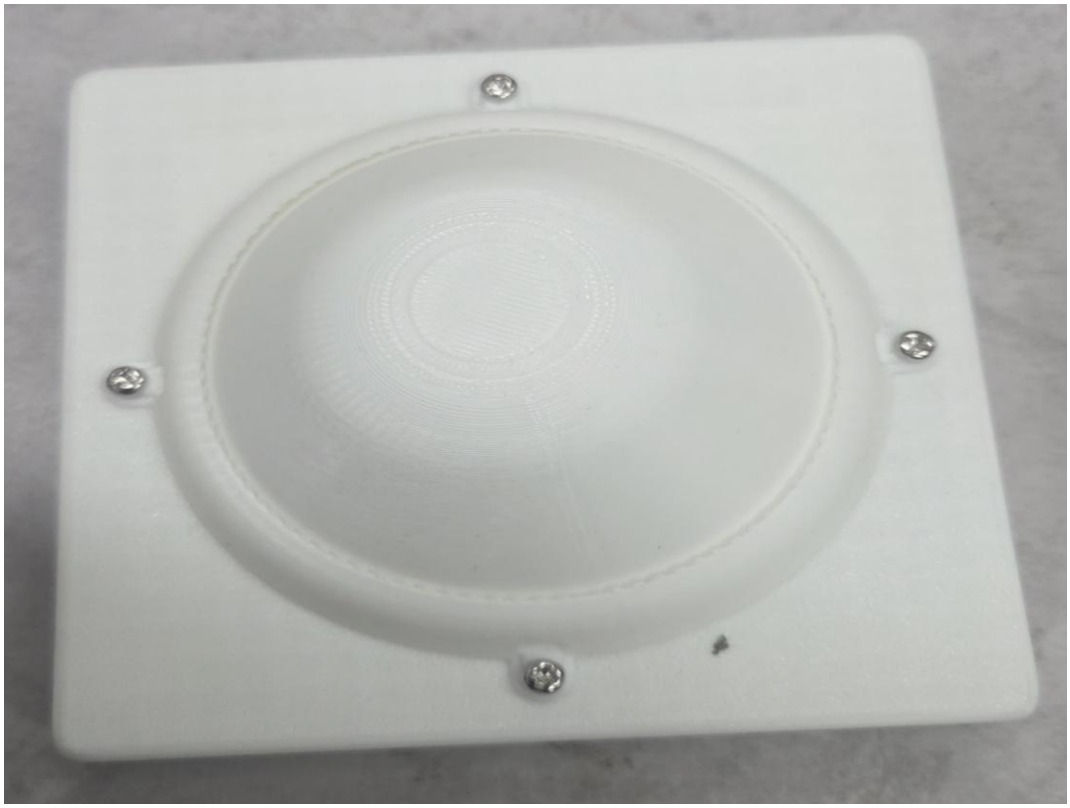


그림 8 다완성 하면 이런 모양입니다

4. Raspberry Pi 에 소스 준비

4.1 GitHub 소스 내려받기

Pi 의 관리자 계정으로 SSH 에 접속한 뒤 다음 명령을 실행한다.

```
sudo apt update
sudo apt install -y git

cd ~
git clone https://github.com/elecage/DU_AT/tree/3767ab7313b188de9ffed64f212d0d080aa1a97b/Carecall_Button/2026_10
cd CaregiverCall-Research
```

다음 명령으로 기준 커밋을 확인한다.

```
git rev-parse HEAD
```

이후 명령에서 사용할 소스 경로를 지정한다.

```
CC_SRC="$"경로""
cd "$CC_SRC"
```

SSH 를 종료하고 다시 접속한 경우에는 CC_SRC=... 명령을 다시 입력한다.

4.2 소스 검사

```
cd "$CC_SRC"
python3 -I -B install/carecall_install_preflight.py
sudo python3 -I -B install/carecall_bootstrap.py plan
```

정상 확인 항목:

```
SOURCE_BASELINE=PASS
BOOTSTRAP=PLAN_ONLY
```

5. Raspberry Pi 기본 환경 설치

5.1 호스트 이름 설정

ESP32 의 기본 MQTT Broker 검색 이름과 맞추기 위해 Pi 호스트 이름을 다음과 같이 설정한다.

```
sudo hostnamectl set-hostname [원하는 이름]
hostnamectl
```

같은 공유기에 [원하는 이름] 라는 부분은 원하는 것으로 이름을 정하고, 사용하는 다른 장치가 없어야 한다.

5.2 필수 패키지 설치

```
sudo apt update
sudo apt install -y ₩
  python3-venv python3-yaml ₩
  mosquitto mosquitto-clients ₩
  avahi-daemon sqlite3 ₩
  hostapd dnsmasq-base wpasupplicant iw netplan.io ₩
  ufw iptables nftables
```

기본 hostapd.service 와 dnsmasq.service 는 직접 활성화하지 않는다. CareCall 전용 서비스가 필요한 때에만 각 프로그램을 실행한다.

다음 명령으로 Pi 가 Wi-Fi 에 연결됐는지 확인한다.

```
ip -4 address show wlan0
ip -4 route show default
systemctl is-active systemd-networkd.service
```

/etc/netplan/50-cloud-init.yaml 은 다음 조건을 만족해야 한다.

- 최상위 항목은 network 하나이다.
- version: 2 를 사용한다.
- 렌더러는 networkd 이다.
- ethernets 에는 eth0 이 있다.
- wifis 에는 wlan0 하나가 있다.
- wlan0 은 dhcp4: true 이다.
- 초기 공유기 한 개만 access-points 에 등록한다.

Netplan 문법 확인:

```
sudo netplan generate
```

6. Raspberry Pi 프로그램과 초기 설정 배치

아래 명령은 모두 GitHub 의 Carecall_Button/2026_10/RaspberryPi 폴더에서 실행한다.

```
cd "$CC_SRC"
```

6.1 새 Pi 준비 가능 여부 확인

```
sudo python3 -I -B install/carecall_prepare_new_pi.py
```

정상 결과:

```
NEW_PI_PREPARE=READY_NO_WRITE
OS_PACKAGES_MISSING=NONE
```

6.2 계정·프로그램·서비스 파일·Python 환경 배치

```
sudo python3 -I -B install/carecall_prepare_new_pi.py ₩
  --apply --confirm PREPARE_NEW_CARECALL_PI
```

정상 결과:

```
NEW_PI_PREPARE=FILES_PREPARED_SERVICES_NOT_ACTIVATED
```

이 명령은 다음 작업을 수행한다.

- carecallserver 계정 생성
- carecall-wifi-web 비로그인 계정 생성
- 서버 프로그램을 /home/carecallserver/carecall_receiver 에 배치
- Wi-Fi 관리자 프로그램을 /opt/carecall-wifi-manager 에 배치
- systemd 서비스 파일 배치
- Python 가상환경 생성
- paho-mqtt 2.1.0 설치

6.3 Telegram Bot 생성

휴대전화의 Telegram 에서 @BotFather 를 연다.

1. /newbot 을 입력한다.
2. Bot 표시 이름을 입력한다.
3. 마지막이 bot 으로 끝나는 Bot 사용자명을 입력한다.
4. 발급된 Bot Token 을 안전하게 보관한다.
5. /setjoingroups 를 입력한다.
6. 생성한 Bot 을 선택하고 그룹 참여를 Disable 로 설정한다.

6.4 MQTT 와 Telegram 기본 설정 생성

먼저 쓰기 없는 점검을 실행한다.

```
sudo python3 -I -B install/carecall_bootstrap.py configure-core
```

정상 결과:

```
BOOTSTRAP=READY_NO_WRITE
```

실제 설정을 생성한다.

```
sudo python3 -I -B install/carecall_bootstrap.py configure-core ₩
  --apply --confirm CREATE_NEW_CARECALL_CORE_CONFIG
```

화면에 다음 값을 순서대로 입력한다. 입력값은 화면에 표시되지 않는다.

1. carecall_receiver MQTT 비밀번호
2. button01 MQTT 비밀번호
3. button02 MQTT 비밀번호
4. button03 MQTT 비밀번호
5. Telegram Bot 사용자명 (@ 제외)
6. Telegram Bot Token

각 비밀번호는 확인을 위해 두 번 입력한다. 네 MQTT 비밀번호는 서로 달라야 한다.

정상 결과:

```
BOOTSTRAP=CORE_CONFIGURED_LISTENER_STAGED
```

6.5 Pi 설정 AP 와 초기 공유기 프로필 생성

```
sudo python3 -I -B install/carecall_bootstrap.py configure-wifi
```

정상 결과:

```
BOOTSTRAP=READY_NO_WRITE
```

실제 설정을 생성한다.

```
sudo python3 -I -B install/carecall_bootstrap.py configure-wifi ₩
  --apply --confirm CREATE_NEW_CARECALL_WIFI_CONFIG
```

입력 항목:

1. CareCall-Pi-로 시작하는 AP 이름
2. 영문·숫자 16~63 자의 Pi 설정 AP 비밀번호

정상 결과:

```
BOOTSTRAP=WIFI_CONFIGURED_NOT_ACTIVATED
```

6.6 빈 데이터베이스 생성

```
sudo python3 -I -B install/carecall_bootstrap.py create-db
```

정상 결과:

```
BOOTSTRAP=READY_NO_WRITE
```

실제 DB 를 생성한다.

```
sudo python3 -I -B install/carecall_bootstrap.py create-db ₩
  --apply --confirm CREATE_NEW_CARECALL_DATABASE
```

정상 결과:

```
BOOTSTRAP=EMPTY_DATABASE_CREATED_DEVICES_STAGED
```

DB 에는 set01 과 button01~button03 이 생성된다. 장치 상태는 아직 staged 이다.

7. UFW 방화벽 설정

7.1 현재 공유기 주소 대역 확인

```
ip -4 route show dev wlan0
```

출력에서 wlan0 에 직접 연결된 사설 주소 대역을 확인한다. 예를 들어 다음과 같이 표시될 수 있다.

```
192.168.0.0/24 proto kernel scope link src 192.168.0.10
```

확인한 주소 대역을 변수에 입력한다. 아래 예시는 실제 환경에 맞게 바꾼다.

```
CC_LAN='192.168.0.0/24'
printf 'CareCall LAN: %s₩n' "$CC_LAN"
```

사용 가능한 대역은 다음 사설 IPv4 범위 안의 단일 대역이다.

- 10.0.0.0/8
- 172.16.0.0/12
- 192.168.0.0/16

Pi 설정 AP 의 192.168.77.0/24 또는 ESP32 설정 AP 의 192.168.78.0/24 와 겹치는 공유기 대역은 사용하지 않는다.

7.2 방화벽 규칙 적용

아래 블록을 한 번에 실행한다. 실행 전 CC_LAN 값이 현재 공유기 대역인지 다시 확인한다.

```
(
  set -e

  sudo sed -i 's/^IPV6=.*/IPV6=yes/' /etc/default/ufw
  sudo ufw --force reset
  sudo ufw default deny incoming
  sudo ufw default allow outgoing
  sudo ufw default deny routed

  sudo ufw allow in on wlan0 proto tcp ₩
    from 192.168.77.0/24 to 192.168.77.1 port 8080 ₩
    comment 'CareCall AP trial web'

  sudo ufw allow in on wlan0 proto udp ₩
    from 0.0.0.0/32 port 68 to 255.255.255.255 port 67 ₩
    comment 'CareCall AP trial DHCP initial'

  sudo ufw allow in on wlan0 proto udp ₩
    from 192.168.77.0/24 port 68 to 255.255.255.255 port 67 ₩
    comment 'CareCall AP trial DHCP broadcast'

  sudo ufw allow in on wlan0 proto udp ₩
    from 192.168.77.0/24 port 68 to 192.168.77.1 port 67 ₩
    comment 'CareCall AP trial DHCP unicast'

  sudo ufw allow in on wlan0 proto tcp ₩
    from "$CC_LAN" to any port 22 ₩
    comment 'CareCall managed LAN SSH'

  sudo ufw allow in on wlan0 proto tcp ₩
    from "$CC_LAN" to any port 1883 ₩
    comment 'CareCall managed LAN MQTT'

  sudo ufw --force enable
)
```

적용 결과 확인:

```
sudo ufw status numbered
```

필수 허용 대상은 다음과 같다.

- 현재 공유기 대역 → Pi TCP 22: SSH
- 현재 공유기 대역 → Pi TCP 1883: MQTT

- Pi 설정 AP 대역 → Pi TCP 8080: Wi-Fi 설정 페이지
- Pi 설정 AP 의 DHCP 요청 → UDP 67

8. 초기 설정 점검

로컬 설정을 검사한다.

```
cd "$CC_SRC"
sudo python3 -I -B install/carecall_bootstrap.py check
```

Telegram API 까지 확인한다.

```
sudo python3 -I -B install/carecall_bootstrap.py check --online
```

정상 확인 항목:

```
CORE_FILES=PASS
WIFI_FILES=PASS
DATABASE_FILES=PASS
MQTT_INITIAL_CONFIG=PASS_LISTENER_STAGED
EMPTY_DATABASE_AND_STAGED_DEVICES=PASS
NETPLAN_AND_PROFILE=PASS
WLAN0_AP_CAPABILITY=YES
NETWORK_SERVICES=PASS
FIREWALL_CURRENT_LAN_RULESET=PASS
MDNS_HOSTNAME_MATCH=YES
TELEGRAM_API_VERIFICATION=PASS
NEW_PI_LOCAL_CHECK=PASS_PRE_ACTIVATION_ONLY
```

REVIEW_REQUIRED 가 나오면 다음 단계로 넘어가지 않고 해당 항목을 확인한다.

9. MQTT listener 와 CareCall 서비스 활성화

9.1 MQTT listener 적용

```
sudo install -o root -g root -m 0644 ₩
  /etc/carecall/initial/mosquitto-carecall.conf ₩
  /etc/mosquitto/conf.d/carecall.conf

sudo systemctl enable --now mosquitto.service
sudo systemctl restart mosquitto.service
sudo systemctl --no-pager --full status mosquitto.service
```

정상 상태:

```
Active: active (running)
```

MQTT 1883 listener 확인:

```
sudo ss -lntp | grep ':1883'
```

9.2 기본 장치 활성화

```
CC_APP='/home/carecallserver/carecall_receiver'

sudo -u carecallserver "$CC_APP/.venv/bin/python" -B ₩
  "$CC_APP/device_admin.py" activate-device --device-id button01

sudo -u carecallserver "$CC_APP/.venv/bin/python" -B ₩
  "$CC_APP/device_admin.py" activate-device --device-id button02

sudo -u carecallserver "$CC_APP/.venv/bin/python" -B ₩
  "$CC_APP/device_admin.py" activate-device --device-id button03
```

상태 확인:

```
sudo -u carecallserver "$CC_APP/.venv/bin/python" -B ₩
  "$CC_APP/device_admin.py" status
```

button01~button03 의 상태가 active 인지 확인한다.

9.3 호출·Telegram 서비스 시작

```
sudo systemctl daemon-reload
sudo systemctl enable --now avahi-daemon.service
sudo systemctl enable --now carecall-receiver.service
sudo systemctl enable --now carecall-telegram.service
sudo systemctl enable --now carecall-registration.service
```

상태 확인:

```
systemctl is-active ₩
  mosquitto.service ₩
  avahi-daemon.service ₩
  carecall-receiver.service ₩
  carecall-telegram.service ₩
  carecall-registration.service
```

모든 줄이 active 인지 확인한다.

10. Pi Wi-Fi 관리자 최초 활성화

10.1 현재 LAN 방화벽 상태 기록 생성

다음 명령은 현재 wlan0 의 사설 주소 대역과 UFW 규칙이 일치하는지 확인한 후 Wi-Fi 관리자가 사용할 상태 기록을 만든다.

```
sudo env CC_SRC="$CC_SRC" python3 -I -B - <<'PY'
import json
import os
import sys
from pathlib import Path

source = Path(os.environ['CC_SRC']).resolve()
sys.path.insert(0, str(source / 'install'))
sys.path.insert(0, str(source / 'wifi-manager'))

import firewall
import firewall_base
from carecall_bootstrap_check import expected_current_rules

state = firewall.state_path()
if state.exists() or state.is_symlink():
    raise SystemExit('LAN_FIREWALL_STATE_ALREADY_EXISTS')

network = firewall.current_network()
records = firewall_base.normalize(firewall_base.run_ufw('show', 'raw'))
expected = expected_current_rules(firewall_base, firewall, network)
if records != expected:
    raise SystemExit('FIREWALL_RULESET_REQUIRES_REVIEW')

firewall.save_state('committed', [network], network)
firewall.inspect()
print('LAN_FIREWALL_INITIAL_STATE=CREATED')
print('LAN_ALLOWED_SUBNET=' + network)
PY
```

정상 결과:

```
LAN_FIREWALL_INITIAL_STATE=CREATED
```

10.2 최초 활성화 기준 파일 생성

다음 명령은 배치된 Wi-Fi 관리자 코드·서비스 파일·현재 Netplan·초기 프로필의 SHA-256 을 기록한다.

```
sudo python3 -I -B - <<'PY'
import hashlib
import json
import os
from pathlib import Path

files = [
    Path('/etc/netplan/50-cloud-init.yaml'),
    Path('/etc/carecall-wifi-manager/settings.json'),
    Path('/etc/carecall-wifi-manager/profile.json'),
    Path('/etc/systemd/system/carecall-wifi-manager.service'),
    Path('/etc/systemd/system/carecall-wifi-manager-hostapd.service'),
    Path('/etc/systemd/system/carecall-wifi-manager-dhcp.service'),
    Path('/etc/systemd/system/carecall-wifi-manager-web.service'),
    Path('/etc/systemd/system/carecall-wifi-manager-router.service'),
    Path('/etc/systemd/system/carecall-wifi-manager.service.d/60-carecall-lan.conf'),
]
files += [
    Path('/opt/carecall-wifi-manager/common.py'),
    Path('/opt/carecall-wifi-manager/firewall.py'),
    Path('/opt/carecall-wifi-manager/firewall_base.py'),
    Path('/opt/carecall-wifi-manager/manager.py'),
    Path('/opt/carecall-wifi-manager/reviewed_firewall.json'),
    Path('/opt/carecall-wifi-manager/router.py'),
    Path('/opt/carecall-wifi-manager/transaction.py'),
    Path('/opt/carecall-wifi-manager/web.py'),
]

hashes = {}
for path in files:
    if not path.is_file() or path.is_symlink():
        raise SystemExit('INVALID_ACTIVATION_FILE=' + str(path))
    info = path.stat()
    if info.st_uid != 0 or info.st_mode & 0o022:
        raise SystemExit('UNTRUSTED_ACTIVATION_FILE=' + str(path))
    hashes[str(path)] = hashlib.sha256(path.read_bytes()).hexdigest()

target = Path('/etc/carecall-wifi-manager/install-baseline.json')
flags = os.O_WRONLY | os.O_CREAT | os.O_EXCL | os.O_NOFOLLOW
fd = os.open(target, flags, 0o600)
with os.fdopen(fd, 'w', encoding='utf-8') as stream:
```

```
    json.dump({'activation_hashes': hashes}, stream, sort_keys=True)
    stream.write('₩n')
    stream.flush()
    os.fsync(stream.fileno())

print('WIFI_INSTALL_BASELINE=CREATED')
print('ACTIVATION_FILE_COUNT=' + str(len(hashes)))
PY
```

정상 결과:

```
WIFI_INSTALL_BASELINE=CREATED
```

10.3 Wi-Fi 관리자 활성화

```
sudo systemctl daemon-reload
sudo python3 -B /opt/carecall-wifi-manager/manager.py activate
```

정상 출력에는 다음 내용이 포함된다.

```
WIFI_PERSIST_ACTIVATION=SCHEDULED
SSH_MAY_DISCONNECT=YES
```

Wi-Fi 관리자가 시작되면 SSH 가 잠시 끊길 수 있다. 약 1 분 뒤 현재 Pi 주소 또는 다음 mDNS 이름으로 다시 접속한다.

```
carecall-mqtt-server.local
```

재접속 후 상태 확인:

```
sudo python3 -B /opt/carecall-wifi-manager/manager.py status
```

주요 정상 항목:

```
MANAGER_SERVICE=active
BOOT_AUTOSTART_ENABLED=YES
PHASE=online
LINK_READY=True
PERSIST_RESULT=COMMITTED
FIREWALL_PRECHECK=PASS
LAN_FIREWALL_PHASE=committed
LAN_RULES_MATCH_CURRENT_NETWORK=YES
```

Pi 를 한 번 재부팅한다.

```
sudo reboot
```

재접속 후 다시 확인한다.

```
sudo python3 -B /opt/carecall-wifi-manager/manager.py status
systemctl is-active ₩
  mosquitto.service ₩
```

```
carecall-receiver.service ₩
carecall-telegram.service ₩
carecall-registration.service ₩
carecall-wifi-manager.service
```

11. Telegram 운영자와 보호자 등록

11.1 운영자 등록

Pi 에서 다음 명령을 실행한다.

```
CC_APP='/home/carecallserver/carecall_receiver'

sudo -u carecallserver "$CC_APP/.venv/bin/python" -B ₩
  "$CC_APP/operator_admin.py" setup
```

터미널에 15 분 동안 사용할 수 있는 Telegram 링크가 표시된다.

1. 운영자로 사용할 휴대전화에서 링크를 연다.
2. Telegram 에서 시작을 누른다.
3. Bot 이 보내는 6 자리 확인 숫자를 확인한다.
4. Pi 터미널에서 다음 명령을 실행한다.

```
sudo -u carecallserver "$CC_APP/.venv/bin/python" -B ₩
  "$CC_APP/operator_admin.py" confirm
```

표시되지 않는 숨김 입력란에 휴대전화의 6 자리 숫자를 입력한다.

정상 결과:

```
OPERATOR_REGISTRATION=SUCCESS
```

상태 확인:

```
sudo -u carecallserver "$CC_APP/.venv/bin/python" -B ₩
  "$CC_APP/operator_admin.py" status
```

정상 확인:

```
operator_configured=true
```

11.2 보호자 등록

운영자 휴대전화에서 CareCall Bot 에 다음 명령을 보낸다.

```
/admin
```

보호자 등록 순서:

1. 보호자 초대 버튼을 누른다.
2. 보호자 별칭을 입력한다.
3. Bot 이 만든 15 분 유효 초대 링크를 해당 보호자에게 전달한다.
4. 보호자가 링크를 열고 시작을 누른다.
5. 보호자 화면에 나타난 6 자리 숫자를 운영자에게 확인시킨다.
6. 운영자는 등록 대기에서 해당 보호자를 선택한다.
7. 확인 숫자 입력 후 승인을 누르고 6 자리 숫자를 입력한다.
8. 등록을 승인했습니다. 메시지를 확인한다.

운영자 본인도 호출 알림을 받아야 한다면 운영자 계정으로 보호자 초대 절차를 한 번 진행한다.

12. ESP32-C3 펌웨어 준비

12.1 Windows 개발 환경

- ESP-IDF v5.4.1 을 설치한다.
- VS Code 의 ESP-IDF 터미널 또는 ESP-IDF PowerShell 을 사용한다.
- ESP32-C3 Super Mini 를 데이터 통신이 가능한 USB 케이블로 PC 에 연결한다.
- Windows 장치 관리자에서 COM 포트를 확인한다.

12.2 장치별 프로젝트 폴더 복사

PowerShell 예시:

```
New-Item -ItemType Directory -Force 'C:₩CareCall_Build' | Out-Null

Copy-Item -Recurse ₩
  'C:₩[자신의 경로] ' ₩
  'C:₩CareCall_Build₩button01'
```

button02, button03 도 같은 방법으로 별도 폴더를 만든다. 장치별 폴더를 분리하면 MQTT 계정이나 Device ID 가 다른 펌웨어와 섞이는 것을 막을 수 있다.

12.3 로컬 설정 파일 생성

button01 폴더에서 다음 명령을 실행한다.

```
Set-Location 'C:₩CareCall_Build₩button01'
Copy-Item '.₩sdkconfig.defaults.example' '.₩sdkconfig.defaults'
```

C:₩CareCall_Build₩button01₩sdkconfig.defaults 를 열고 다음 항목을 설정한다.

```
CONFIG_CARECALL_DEVICE_ID="button01"
CONFIG_CARECALL_MQTT_USERNAME="button01"
CONFIG_CARECALL_MQTT_PASSWORD="Pi 에서 button01 에 지정한 MQTT 비밀번호"
CONFIG_CARECALL_MQTT_CLIENT_ID="carecall-button01"
CONFIG_CARECALL_MQTT_BROKER_MDNS_HOST="carecall-mqtt-server"
CONFIG_CARECALL_MQTT_BROKER_PORT=1883
CONFIG_CARECALL_SETUP_AP_PASSWORD="ESP32 설정 AP 고정 비밀번호"
CONFIG_CARECALL_WIFI_SSID=""
CONFIG_CARECALL_WIFI_PASSWORD=""
CONFIG_CARECALL_LED_GPIO=2
CONFIG_CARECALL_LED_BRIGHTNESS=16
```

Wi-Fi SSID 와 비밀번호는 휴대전화 설정 페이지에서 입력하므로 기본적으로 빈 값으로 둔다.

button02 와 button03 은 장치별로 다음 세 항목을 바꾼다.

장치	Device ID	MQTT 사용자명	Client ID
button01	button01	button01	carecall-button01
button02	button02	button02	carecall-button02
button03	button03	button03	carecall-button03

각 장치의 CONFIG_CARECALL_MQTT_PASSWORD 에는 Pi 설정 시 해당 계정에 지정한 비밀번호를 입력한다.

13. ESP32-C3 빌드 및 플래시

button01 의 ESP-IDF PowerShell 에서 실행하는 예시이다. COM5 는 실제 포트로 바꾼다.

```
Set-Location 'C:₩CareCall_Build₩button01'

idf.py set-target esp32c3
idf.py reconfigure
idf.py build
idf.py -p COM5 erase-flash
```

```
idf.py -p COM5 flash monitor
```

정상적으로 기록되면 ESP32-C3 가 재부팅되고 모니터에 CareCall 시작 로그가 표시된다.

모니터 종료:

```
Ctrl + ]
```

button02 와 button03 도 각각의 프로젝트 폴더와 COM 포트에서 같은 방식으로 빌드하고 기록한다.

비밀값이 들어간 다음 파일은 GitHub 에 올리지 않는다.

- sdkconfig
- 실제 값이 들어간 sdkconfig.defaults
- build/
- .bin, .elf, .map

14. ESP32-C3 최초 Wi-Fi 설정

저장된 Wi-Fi 가 없는 새 ESP32-C3 는 전원을 켜면 설정 AP 를 연다.

AP 이름 형식:

```
CareCall-ESP32-XXXXXX
```

휴대전화에서 다음 순서로 진행한다.

1. CareCall-ESP32-XXXXXX Wi-Fi 에 연결한다.
2. 펌웨어 빌드 때 입력한 ESP32 설정 AP 비밀번호를 입력한다.
3. 모바일 브라우저에서 http://192.168.78.1:8080 을 연다.
4. Pi 와 같은 2.4 GHz 공유기의 SSID 와 비밀번호를 입력한다.
5. 입력한 Wi-Fi 연결 확인을 누른다.
6. TEST_OK 또는 연결 확인 안내가 표시되면 이 Wi-Fi 로 저장하고 연결을 누른다.
7. 설정 AP 가 종료되고 ESP32-C3 가 공유기에 연결되는지 확인한다.

Pi 와 ESP32-C3 는 같은 로컬 네트워크에 연결되어야 하며 공유기의 단말 격리 기능이 꺼져 있어야 한다.

15. 전체 기능 시험

15.1 서비스 상태 확인

Pi 에서 실행한다.

```
systemctl is-active ₩
  mosquitto.service ₩
  avahi-daemon.service ₩
  carecall-receiver.service ₩
  carecall-telegram.service ₩
  carecall-registration.service ₩
  carecall-wifi-manager.service
```

모두 active 인지 확인한다.

최근 로그 확인:

```
sudo journalctl -u carecall-receiver.service -n 80 --no-pager
sudo journalctl -u carecall-telegram.service -n 80 --no-pager
sudo journalctl -u carecall-registration.service -n 80 --no-pager
```

15.2 호출·확인 시험

1. ESP32-C3 의 호출 버튼을 한 번 누른다.
2. NeoPixel LED 링이 켜지는지 확인한다.
3. Telegram 보호자 계정에 새 호출 알림이 오는지 확인한다.
4. 알림에 확인했습니다. 버튼이 생기는지 확인한다.
5. 보호자 한 명이 확인했습니다.를 누른다.
6. ESP32-C3 의 LED 가 꺼지는지 확인한다.
7. Pi 로그에서 MQTT 수신, DB 저장, Application ACK, Telegram 전송을 확인한다.

15.3 최신 호출 시험

1. 첫 호출 알림을 받은 뒤 다시 버튼을 눌러 두 번째 호출을 발생시킨다.
2. 이전 알림의 확인 버튼이 제거되는지 확인한다.
3. 최신 알림의 확인했습니다. 버튼만 정상 처리되는지 확인한다.
4. 최신 호출을 확인했을 때 LED 가 꺼지는지 확인한다.

15.4 통신 복구 시험

1. ESP32-C3 의 Wi-Fi 연결을 일시적으로 끊는다.
2. 오프라인 상태에서 호출 버튼을 누른다.
3. LED 가 켜지고 호출이 장치에 보존되는지 확인한다.
4. Wi-Fi 를 복구한다.
5. 저장된 호출이 자동으로 Pi 에 전송되고 Telegram 알림이 오는지 확인한다.
6. 보호자 확인 후 LED 가 꺼지는지 확인한다.

16. 설치 장소에서 Wi-Fi 변경

16.1 Raspberry Pi Wi-Fi 변경

Pi 가 현재 공유기에 정상 연결된 상태에서 다음 명령을 실행한다.

```
sudo python3 -B /opt/carecall-wifi-manager/manager.py setup
```

SSH 가 끊긴 뒤 휴대전화에서 다음 순서로 진행한다.

1. 설치할 때 지정한 CareCall-Pi-... Wi-Fi 에 연결한다.
2. Pi 설정 AP 비밀번호를 입력한다.
3. 브라우저에서 http://192.168.77.1:8080 을 연다.
4. 새 공유기의 SSID 와 비밀번호를 입력한다.
5. 연결 시험을 진행한다.
6. 연결 성공 안내 후 저장·확정 버튼을 누른다.
7. Pi 설정 AP 가 종료되고 Pi 가 새 공유기에 연결될 때까지 기다린다.

새 공유기에 연결되면 Pi 주소가 바뀔 수 있다. 공유기 관리 화면 또는 carecall-mqtt-server.local 로 다시 접속한다.

상태 확인:

```
sudo python3 -B /opt/carecall-wifi-manager/manager.py status
```

16.2 ESP32-C3 Wi-Fi 변경

1. ESP32-C3 의 전원을 끈다.
2. 호출 버튼을 누른 상태로 전원을 켠다.
3. 호출 버튼을 3 초 이상 계속 누른 뒤 놓는다.
4. 휴대전화에서 CareCall-ESP32-XXXXXX Wi-Fi 에 연결한다.
5. 브라우저에서 http://192.168.78.1:8080 을 연다.
6. 새 공유기 정보를 입력하고 연결 확인을 누른다.
7. 연결 성공 후 저장한다.

여러 ESP32-C3 가 있으면 각 장치에서 위 절차를 반복한다.

17. 호출 버튼 추가

다음 예시는 button04 를 set01 에 추가하는 방법이다.

17.1 MQTT 계정 추가

```
sudo mosquitto_passwd /etc/mosquitto/passwd button04
sudo chown root:mosquitto /etc/mosquitto/passwd
sudo chmod 0640 /etc/mosquitto/passwd
```

새 비밀번호를 숨김 입력하고 안전하게 보관한다.

17.2 MQTT ACL 추가

```
sudoedit /etc/mosquitto/acl
```

파일 마지막에 다음 내용을 추가한다.

```
user button04
topic write carecall/v1/devices/button04/call
topic read carecall/v1/devices/button04/ack
```

권한을 확인한다.

```
sudo chown root:mosquitto /etc/mosquitto/acl
sudo chmod 0640 /etc/mosquitto/acl
```

17.3 DB 에 장치 등록

```
CC_APP='/home/carecallserver/carecall_receiver'

sudo -u carecallserver "$CC_APP/.venv/bin/python" -B ₩
  "$CC_APP/device_admin.py" stage-device ₩
  --device-id button04 --set-id set01

sudo -u carecallserver "$CC_APP/.venv/bin/python" -B ₩
  "$CC_APP/device_admin.py" activate-device ₩
  --device-id button04
```

17.4 서비스 반영

```
sudo systemctl restart mosquitto.service
sudo systemctl restart carecall-receiver.service
```

17.5 ESP32-C3 설정

새 ESP32 프로젝트에서 다음 값을 사용한다.

```
CONFIG_CARECALL_DEVICE_ID="button04"
CONFIG_CARECALL_MQTT_USERNAME="button04"
CONFIG_CARECALL_MQTT_PASSWORD="button04 에 새로 지정한 MQTT 비밀번호"
CONFIG_CARECALL_MQTT_CLIENT_ID="carecall-button04"
```

빌드·플래시·Wi-Fi 설정 후 호출 시험을 진행한다.

18. 장애 확인 명령

18.1 전체 서비스 상태

```
systemctl --no-pager --full status ₩
  mosquitto.service ₩
  carecall-receiver.service ₩
  carecall-telegram.service ₩
  carecall-registration.service ₩
  carecall-wifi-manager.service
```

18.2 최근 로그

```
sudo journalctl -u mosquitto.service -n 100 --no-pager
sudo journalctl -u carecall-receiver.service -n 100 --no-pager
sudo journalctl -u carecall-telegram.service -n 100 --no-pager
sudo journalctl -u carecall-registration.service -n 100 --no-pager
```

```
sudo journalctl -u carecall-wifi-manager.service -n 100 --no-pager
```

18.3 장치 등록 상태

```
CC_APP='/home/carecallserver/carecall_receiver'

sudo -u carecallserver "$CC_APP/.venv/bin/python" -B ₩
  "$CC_APP/device_admin.py" status
```

18.4 운영자·보호자 등록 상태

```
sudo -u carecallserver "$CC_APP/.venv/bin/python" -B ₩
  "$CC_APP/operator_admin.py" status
```

18.5 Pi Wi-Fi 관리자 상태

```
sudo python3 -B /opt/carecall-wifi-manager/manager.py status
sudo ufw status numbered
ip -4 address show wlan0
ip -4 route show default
```